

TaskFlow API

التقرير الفني للمشروع

مادة الحوسبة السحابية

نظام إدارة المهام باستخدام واجهة برمجة التطبيقات RESTful

اعداد الطالبات:

اميمة محمود غيلان

عبير علي الرميلى

اسماد احمد الرعيني

امة الرحمن سمير النجار

الملخص:

يهدف مشروع **TaskFlow API** إلى تطوير نظام لإدارة المهام باستخدام واجهة برمجة التطبيقات **RESTful API** ، مع تطبيق مفاهيم الحوسبة السحابية والحاويات باستخدام **Docker** و **Docker Compose** .

يتكون النظام من ثلاثة مكونات رئيسية: خادم **Nginx** يعمل كـ **Reverse Proxy** ، وتطبيق **FastAPI** لمعالجة طلبات الـ **API** ، وقاعدة بيانات **PostgreSQL** لتخزين بيانات المهام. تتواصل الخدمات من خلال شبكة **Docker** داخلية، بينما يتم نشر منفذ **Nginx** فقط على الجهاز المضيف.

يوفر النظام عمليات إدارة كاملة للمهام تشمل الإضافة والعرض والتعديل والحذف **CRUD** ، بالإضافة إلى تخزين البيانات بشكل دائم باستخدام **Named Docker Volume**. كما تم تطبيق تشغيل حاوية الـ **API** باستخدام مستخدم غير **root** ، وإضافة فحص لصحة قاعدة البيانات، واختبار عزل المنافذ.

تم كذلك اختبار حالات الفشل والاستعادة، بما في ذلك توقف خدمة الـ **API** وتوقف قاعدة البيانات وإعادة تشغيل الخدمات، بالإضافة إلى التحقق من استمرار البيانات بعد إعادة إنشاء الحاويات. وتم نشر صورة الـ **API** بإصدار محدد على **Docker Hub** لدعم قابلية إعادة الإنتاج والنشر.

المقدمة:

أصبحت تقنيات الحوسبة السحابية والحاويات من الأساليب المهمة في تطوير وتشغيل التطبيقات، لما توفره من بيئات تشغيل معزولة وقابلة لإعادة الاستخدام والنشر. ويُعد **Docker** من التقنيات المستخدمة لتغليف التطبيقات مع مكوناتها وبيئة تشغيلها داخل حاويات مستقلة، بينما يساعد **Docker Compose** في إدارة وتشغيل مجموعة من الخدمات التي تعمل معًا ضمن نظام واحد.

في هذا المشروع تم تطوير TaskFlow API كنظام لإدارة المهام يعتمد على واجهة برمجة التطبيقات RESTful API. تم بناء التطبيق باستخدام FastAPI ، مع استخدام PostgreSQL لتخزين البيانات، و Nginx كـ Reverse Proxy لاستقبال الطلبات وتوجيهها إلى خدمة الـ API.

تم تصميم النظام بحيث تعمل الخدمات داخل حاويات منفصلة وتتواصل من خلال شبكة Docker داخلية، مع جعل Nginx هو الخدمة الوحيدة التي يمكن الوصول إليها من الجهاز المضيف. كما تم استخدام Named Volume لحفظ بيانات قاعدة البيانات واستمرارها عند إيقاف الحاويات أو إعادة إنشائها.

يركز المشروع على تطبيق مفاهيم الحاويات بصورة عملية، بما في ذلك Process Isolation، و Network Isolation، و Port Isolation، وتشغيل تطبيق الـ API باستخدام مستخدم غير root ، بالإضافة إلى استخدام Health Check لقاعدة البيانات واختبار حالات الفشل والاستعادة. كما تم إنشاء صورة Docker بإصدار محدد ونشرها على Docker Hub لدعم قابلية إعادة الإنتاج والنشر.

اهداف المشروع:

يهدف مشروع TaskFlow API إلى تطبيق مفاهيم الحوسبة السحابية والحاويات من خلال بناء نظام عملي لإدارة المهام، وتحقيق الأهداف التالية:

- بناء واجهة برمجة تطبيقات RESTful API لإدارة المهام.
- تطبيق عمليات CRUD الأساسية: الإضافة، العرض، التعديل، والحذف.
- تشغيل مكونات النظام داخل حاويات Docker مستقلة.
- استخدام Nginx كـ Reverse Proxy أمام خدمة الـ API.

- تحقيق عزل الخدمات من خلال شبكة Docker داخلية.
- منع الوصول المباشر إلى منفذ خدمة الـ API من الجهاز المضيف.
- استخدام PostgreSQL لتخزين بيانات النظام.
- ضمان استمرار بيانات قاعدة البيانات باستخدام Named Volume.
- تشغيل خدمة الـ API باستخدام مستخدم غير root.
- استخدام Docker Compose لإدارة وتشغيل خدمات النظام بطريقة declarative.
- اختبار عمل النظام في الظروف الطبيعية وحالات الفشل والاستعادة.
- إنشاء صورة Docker بإصدار محدد ونشرها على Docker Hub لدعم إعادة الإنتاج والنشر.

نظرة عامة على النظام:

TaskFlow API هو نظام لإدارة المهام تم تطويره كخدمة RESTful API ، ويعتمد على مجموعة من الخدمات التي تعمل داخل حاويات Docker

يتكون النظام من ثلاث خدمات رئيسية:

- Nginx: يعمل كـ Reverse Proxy ويستقبل الطلبات القادمة من المستخدم، وهو الخدمة الوحيدة التي يتم نشر منفذها على الجهاز المضيف.
- FastAPI: يمثل خدمة التطبيق والـ API ، ويعالج طلبات المستخدم وينفذ عمليات إدارة المهام.
- PostgreSQL: يمثل خدمة قاعدة البيانات، ويُستخدم لتخزين بيانات المهام.

تتواصل الخدمات من خلال شبكة Docker داخلية، بينما يتم تخزين بيانات PostgreSQL باستخدام Named Volume يسمى db-data ، مما يسمح بالحفاظ على البيانات عند إيقاف الحاويات أو إعادة تشغيلها.

يبدأ تدفق الطلب من المستخدم إلى Nginx ، ثم يتم توجيهه إلى FastAPI ، وبعد ذلك تتواصل خدمة الـ API مع PostgreSQL لتنفيذ عمليات تخزين واسترجاع البيانات.

الآلية عمل النظام:

عند تشغيل المشروع باستخدام Docker Compose ، يتم إنشاء وتشغيل الخدمات الثلاث داخل بيئة حاويات منفصلة. تقوم خدمة Nginx باستقبال طلبات المستخدم من خلال المنفذ **8080**، ثم تمرر الطلبات إلى خدمة FastAPI عبر شبكة Docker الداخلية.

تقوم خدمة FastAPI بمعالجة الطلب وتنفيذ العملية المطلوبة، مثل إنشاء مهمة جديدة أو عرض المهام أو تعديلها أو حذفها. وعند الحاجة إلى التعامل مع البيانات، تتصل خدمة FastAPI بخدمة PostgreSQL باستخدام الاتصال الداخلي بين الحاويات.

يتم تخزين بيانات قاعدة البيانات في Named Volume باسم db-data. وبذلك لا تعتمد البيانات على دورة حياة حاوية PostgreSQL نفسها، مما يساعد على استمرار البيانات عند إيقاف الحاويات أو إعادة تشغيلها.

كما تم تصميم النظام بحيث لا يتم نشر منفذ FastAPI أو PostgreSQL مباشرة على الجهاز المضيف، ويكون الوصول الخارجي إلى النظام من خلال Nginx فقط. وهذا يحقق مبدأ Port Isolation ويدعم فصل الخدمات داخل بيئة Docker.

مكونات النظام:

وملفات قاعدة البيانات والنماذج والمخططات FastAPI ويحتوي على تطبيق **api/**.
Reverse Proxy المستخدمة كـ Nginx ويحتوي على إعدادات **nginx/**.
Named Volume ويحتوي على تعريف الخدمات والشبكة والـ **compose.yaml**.
وطريقة تشغيل النظام.
ويستخدم لحفظ وثائق المشروع والتقارير الفني **docs/**.
للمساعدة في تنظيم الملفات واستبعاد الملفات غير **.dockerignore** و **.gitignore**.
المطلوبة.

معمارية النظام:

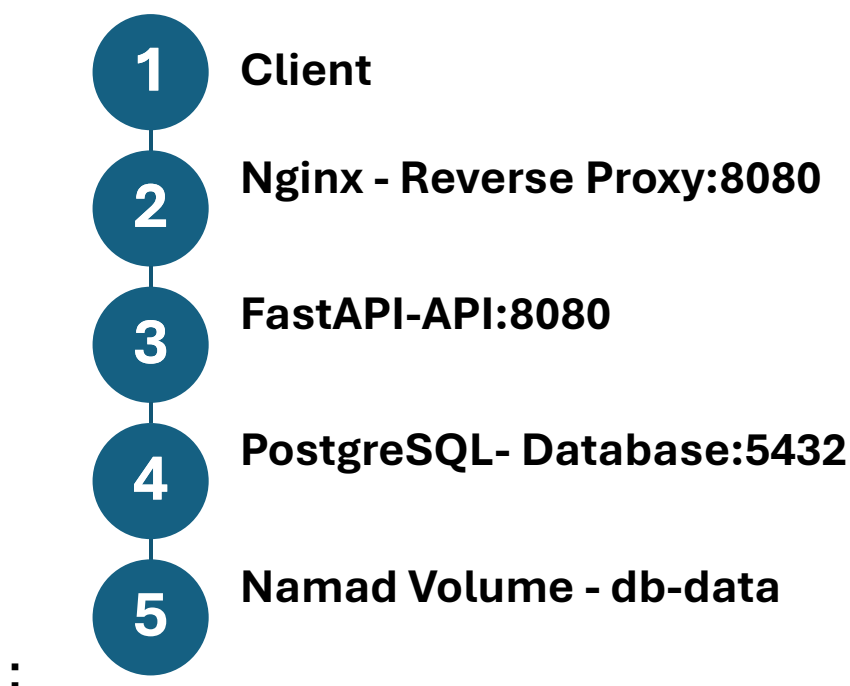
تم تصميم نظام TaskFlow API باستخدام معمارية تعتمد على ثلاث خدمات رئيسية تعمل داخل حاويات Docker منفصلة، وهي Nginx، FastAPI، و PostgreSQL.

تعمل خدمة Nginx ك Reverse Proxy وتستقبل الطلبات القادمة من المستخدم من خلال المنفذ **8080**، ثم تقوم بتوجيهها إلى خدمة FastAPI عبر شبكة Docker الداخلية.

تقوم خدمة FastAPI بمعالجة طلبات الـ API وتنفيذ العمليات المطلوبة على المهام، وتتصل بخدمة PostgreSQL للوصول إلى البيانات وتخزينها.

يتم تخزين بيانات PostgreSQL داخل Named Volume باسم db-data، مما يسمح بالحفاظ على البيانات عند إيقاف أو إعادة تشغيل الحاويات.

مخطط المعمارية:



يوضح المخطط مسار الاتصال داخل النظام، حيث يكون Nginx هو نقطة الدخول الوحيدة من الجهاز المضيف، بينما تعمل خدمة FastAPI وقاعدة البيانات داخل شبكة Docker الداخلية دون نشر منافذها مباشرة على الجهاز المضيف.

```
C:\Users\omimah\Desktop\TaskFlow>docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED          STATUS              PORTS
taskflow-api-1      taskflow-api        "uvicorn src.main:ap..." api         About an hour ago Up 9 minutes        8000/tcp
taskflow-database-1 postgres:16          "docker-entrypoint.s..." database    About an hour ago Up 29 minutes (healthy) 5432/tcp
taskflow-nginx-1    nginx:alpine        "/docker-entrypoint..." nginx       About an hour ago Up About an hour      0.0.0.0:8000->80/tcp, [::]:8000->80/tcp
```

تنشيط Windows
انتقل الى الاعدادات لتنشيط Windows

عزل منافذ الخدمات والتأكد من أن Nginx هو المنفذ المنشور على الجهاز المضيف.

```
{
  "Labels": {
    "com.docker.compose.config-hash": "a49500728875318fb3970d3d27a3b950a030ce0a839a15052135017f98b25eb4",
    "com.docker.compose.network": "app-network",
    "com.docker.compose.project": "taskflow",
    "com.docker.compose.version": "5.1.4"
  },
  "Containers": {
    "3b64a222c05e5e5cb36eb88f3d90e0b1ac76d2b916f8fa42727a1c6b7f2f438c": {
      "Name": "taskflow-database-1",
      "EndpointID": "f29c267402f98dce8146340bab622cee84087dee25045682bfa0d428a8e70497",
      "MacAddress": "22:e3:35:11:ae:09",
      "IPv4Address": "172.18.0.2/16",
      "IPv6Address": ""
    },
    "54cfe17a0f3a2f218572896763b0a18980b28c648a59dfe26c7dad621c3a0563": {
      "Name": "taskflow-nginx-1",
      "EndpointID": "b9e3784e7bd593ba745e726623b38dedf41aca5b7a12588d641bbe58a64a99cc",
      "MacAddress": "e6:ba:f3:24:27:8c",
      "IPv4Address": "172.18.0.4/16",
      "IPv6Address": ""
    },
    "dd7d87a646e1aad096d5a2b31ed6bcdcd7d05b1e21fa01c8c43e54989becf24c6": {
      "Name": "taskflow-api-1",
      "EndpointID": "084136dd1233d6d36227b259064d1e2f575ce3305a4d6abf53b8e2195d3937df",
      "MacAddress": "12:3c:d0:59:2b:24",
      "IPv4Address": "172.18.0.3/16",
      "IPv6Address": ""
    }
  },
  "Status": {
    "IPAM": {
      "Subnets": {
        "172.18.0.0/16": {
          "IPsInUse": 6,
          "DynamicIPsAvailable": 65530
        }
      }
    }
  }
}
```

فحص شبكة Docker الداخلية والتأكد من اتصال الخدمات ضمن الشبكة.

تقنيات الحاويات المستخدمة:

تم استخدام مجموعة من تقنيات وأدوات الحاويات لبناء وتشغيل نظام TaskFlow API ، بحيث تعمل مكونات النظام في بيئة معزولة وقابلة لإعادة التشغيل والنشر.

Docker :

تم استخدام Docker لإنشاء وتشغيل الحاويات الخاصة بمكونات النظام، مما يسمح بتشغيل كل خدمة في بيئة مستقلة عن الخدمات الأخرى.

: Docker Compose

تم استخدام Docker Compose لإدارة الخدمات الثلاث وتشغيلها معاً، وهي:

- Nginx
- FastAPI
- PostgreSQL

كما يحدد ملف [compose.yaml](#) الشبكة الداخلية والـ Named Volume والعلاقات بين الخدمات.

: Docker Network

تم إنشاء شبكة Docker داخلية باسم app-network لتمكين الخدمات من التواصل مع بعضها داخل بيئة الحاويات. ولا تحتاج خدمة FastAPI أو PostgreSQL إلى نشر منافذها مباشرة على الجهاز المضيف.

: Named Volume

تم استخدام Named Volume باسم db-data لتخزين بيانات PostgreSQL بشكل دائم، بحيث تبقى البيانات موجودة حتى بعد إيقاف أو إعادة تشغيل الحاويات.

: Nginx Reverse Proxy

تم استخدام Nginx كـ Reverse Proxy لاستقبال الطلبات من المستخدم وتوجيهها إلى خدمة FastAPI .

ويُعد Nginx نقطة الدخول الخارجية الوحيدة للنظام، بينما تبقى خدمات الـ API وقاعدة البيانات داخل الشبكة الداخلية.

```

C:\Users\omimah\Desktop\TaskFlow>docker compose config
name: taskflow
services:
  api:
    build:
      context: C:\Users\omimah\Desktop\TaskFlow\api
      dockerfile: Dockerfile
    depends_on:
      database:
        condition: service_healthy
        required: true
    environment:
      DATABASE_URL: postgresql://taskflow_user:taskflow_pass@database:5432/taskflow
    networks:
      app-network: null
  database:
    environment:
      POSTGRES_DB: taskflow
      POSTGRES_PASSWORD: taskflow_pass
      POSTGRES_USER: taskflow_user
    healthcheck:
      test:
        - CMD-SHELL
        - pg_isready -U taskflow_user -d taskflow
      timeout: 5s
      interval: 5s
      retries: 5
    image: postgres:16
    networks:
      app-network: null
    volumes:
      - type: volume
        source: db-data
        target: /var/lib/postgresql/data
        volume: {}
  nginx:
    depends_on:
      api:
        condition: service_started
        required: true
    image: nginx:alpine
    networks:
      app-network: null
    ports:
      - mode: ingress
        target: 80
        published: "8080"
        protocol: tcp
    volumes:
      - type: bind
        source: C:\Users\omimah\Desktop\TaskFlow\nginx\nginx.conf
        target: /etc/nginx/nginx.conf
        read_only: true
        bind: {}
networks:
  app-network:
    name: taskflow_app-network
    driver: bridge
volumes:
  db-data:
    name: taskflow_db-data

```

تنشيط Windows
انتقل إلى الإعدادات لتنشيط Windows

التحقق من اعدادات الخدمات باستخدام Docker compose

```

C:\Users\omimah\Desktop\TaskFlow>docker volume ls
DRIVER      VOLUME NAME
local       f530e3705d8657f6957ffc023a882b5dfbb6568ace47afdc46d47358c4717ccb
local       taskflow_db-data

```

C:\Users\omimah\Desktop\TaskFlow>

التحقق من انشاء Named Volume المستخدم لتخزين بيانات قاعدة البيانات

```
C:\Users\omimah\Desktop\TaskFlow>docker volume ls
DRIVER      VOLUME NAME
local       f530e3705d8657f6957ffc023a882b5dfbb6568ace47afdc46d47358c4717ccb
local       taskflow_db-data

C:\Users\omimah\Desktop\TaskFlow>
```

عرض حالة حاويات النظام والتأكد من تشغيل الخدمات

واجهة برمجة التطبيقات CRUD:

GET /api/tasks — عرض جميع المهام.
 GET /api/tasks/{task_id} — عرض مهمة محددة.
 POST /api/tasks — إضافة مهمة جديدة.
 PUT /api/tasks/{task_id} — تعديل مهمة.
 DELETE /api/tasks/{task_id} — حذف مهمة.

تم اختبار عمليات CRUD والتأكد من اتصال الـ API بقاعدة بيانات PostgreSQL.

```
C:\Users\omimah\Desktop\TaskFlow>curl http://127.0.0.1:8080/api/tasks/1
{"title":"TaskFlow Test","description":"Testing Nginx reverse proxy","status":"in_progress","id":1}
C:\Users\omimah\Desktop\TaskFlow>
```

اختبار جلب مهمة محددة باستخدام GET

```
C:\Users\omimah\Desktop\TaskFlow>curl -X PUT http://127.0.0.1:8080/api/tasks/1 -H "Content-Type: application/json" -d '{"title":"TaskFlow Updated","description":"Updated through Nginx","status":"completed","id":1}'
{"title":"TaskFlow Updated","description":"Updated through Nginx","status":"completed","id":1}
C:\Users\omimah\Desktop\TaskFlow>
```

اختبار تعديل بيانات المهمة باستخدام PUT

```
C:\Users\omimah\Desktop\TaskFlow>curl.exe -i -X "DELETE" "http://127.0.0.1:8080/api/tasks/1"
HTTP/1.1 204 No Content
Server: nginx/1.31.6
Date: Mon, 28 Sep 2026 02:02:35 GMT
Content-Type: application/json
Connection: keep-alive

C:\Users\omimah\Desktop\TaskFlow>
```

اختبار حذف مهمة باستخدام DELETE

```
C:\Users\omimah\Desktop\TaskFlow>curl.exe http://127.0.0.1:8080/api/tasks
[]
C:\Users\omimah\Desktop\TaskFlow>
```

التحقق من حذف المهمة بنجاح

قاعدة البيانات واستمرارية البيانات :

تم استخدام PostgreSQL كقاعدة بيانات لنظام TaskFlow API ، ويتم تخزين بيانات المهام داخل قاعدة البيانات.

تم ربط خدمة FastAPI بقاعدة البيانات من خلال شبكة Docker الداخلية، مع استخدام Named Volume باسم db-data لضمان استمرار البيانات.

تم اختبار استمرارية البيانات بعد إيقاف وإعادة تشغيل الخدمات، والتأكد من بقاء البيانات محفوظة.

```
C:\Users\omimah\Desktop\TaskFlow>curl http://127.0.0.1:8080/api/tasks
[{"title":"TaskFlow Test","description":"Testing Nginx reverse proxy","status":"in_progress","id":1}]
C:\Users\omimah\Desktop\TaskFlow>
```

لتحقق من استمرار البيانات من خلال Nginx

```
C:\Users\omimah\Desktop\TaskFlow>curl http://127.0.0.1:8080/api/tasks
[{"title":"TaskFlow Test","description":"Testing Nginx reverse proxy","status":"in_progress","id":1}]
C:\Users\omimah\Desktop\TaskFlow>
```

التحقق من بقاء البيانات بعد إعادة تشغيل الخدمات

```
C:\Users\omimah\Desktop\TaskFlow>docker compose start database
[+] start 1/1
✓Container taskflow-database-1 Started

C:\Users\omimah\Desktop\TaskFlow>docker compose ps

```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
taskflow-api-1	taskflow-api	"uvicorn src.main:ap..."	api	36 minutes ago	Up 8 minutes	8080/tcp
taskflow-database-1	postgres:16	"docker-entrypoint.s..."	database	36 minutes ago	Up 17 seconds (healthy)	5432/tcp
taskflow-nginx-1	nginx:alpine	"/docker-entrypoint..."	nginx	36 minutes ago	Up 36 minutes	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp

```
C:\Users\omimah\Desktop\TaskFlow>curl.exe http://127.0.0.1:8080/api/tasks
[1]
C:\Users\omimah\Desktop\TaskFlow>
```

تنشيط Windows
انتقل إلى الإعدادات لتنشيط Windows.

التحقق من استعادة خدمة قاعدة البيانات وعودة النظام للعمل

الأمان وعزل الخدمات:

تم تطبيق عدد من إجراءات العزل والأمان في نظام TaskFlow API ، أهمها:

- تشغيل خدمة FastAPI باستخدام مستخدم غير root.
- عدم نشر منفذ FastAPI مباشرة على الجهاز المضيف.
- عدم نشر منفذ PostgreSQL مباشرة على الجهاز المضيف.
- استخدام شبكة Docker داخلية لربط الخدمات.
- جعل Nginx نقطة الدخول الوحيدة للنظام.

```
C:\Users\omimah\Desktop\TaskFlow\api>docker run --rm taskflow-api:v1.0.0 id
uid=1000(appuser) gid=1000(appuser) groups=1000(appuser)

C:\Users\omimah\Desktop\TaskFlow\api>
```

لتحقق من تشغيل حاوية الـ API باستخدام مستخدم غير root.

```
C:\Users\omimah\Desktop\TaskFlow>docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED      STATUS        PORTS
taskflow-api-1      taskflow-api        "uvicorn src.main:ap-  api        8 minutes ago Up 8 minutes  8080/tcp
taskflow-database-1 postgres:16          "docker-entrypoint.s-  database   8 minutes ago Up 8 minutes (healthy)  5432/tcp
taskflow-nginx-1    nginx:alpine        "/docker-entrypoint...  nginx      8 minutes ago Up 8 minutes  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
```

التحقق من عزل منفذ خدمة الـ API عن الجهاز المضيف.

اختبار حالات الفشل والاستعادة:

تم اختبار النظام في عدة حالات للتأكد من قدرته على التعامل مع توقف الخدمات واستعادتها.

توقف خدمة API

تم إيقاف خدمة الـ API واختبار الوصول إليها من خلال Nginx، وظهرت استجابة ٥٠٢، ثم تمت إعادة تشغيل الخدمة والتحقق من عودة النظام.

```
C:\Users\omimah\Desktop\TaskFlow>curl.exe -i http://127.0.0.1:8080/api/tasks
HTTP/1.1 502 Bad Gateway
Server: nginx/1.31.6
Date: Mon, 28 Sep 2026 02:08:36 GMT
Content-Type: text/html
Content-Length: 157
Connection: keep-alive

<html>
<head><title>502 Bad Gateway</title></head>
<body>
<center><h1>502 Bad Gateway</h1></center>
<hr><center>nginx/1.31.6</center>
</body>
</html>
تنشيط Windows
انتقل إلى الإعدادات لتنشيط Windows.
C:\Users\omimah\Desktop\TaskFlow>
```

اختبار النظام عند توقف خدمة الـ API

```
C:\Users\omimah\Desktop\TaskFlow>docker compose start api
[+] start 2/2
✓Container taskflow-database-1 Healthy
✓Container taskflow-api-1 Started

C:\Users\omimah\Desktop\TaskFlow>docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS        PORTS
taskflow-api-1      taskflow-api        "uvicorn src.main:ap..." api         28 minutes ago Up 14 seconds 8080/tcp
taskflow-database-1 postgres:16         "docker-entrypoint.s..." database   28 minutes ago Up 28 minutes (healthy) 5432/tcp
taskflow-nginx-1    nginx:alpine        "/docker-entrypoint..." nginx      28 minutes ago Up 28 minutes 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp

C:\Users\omimah\Desktop\TaskFlow>curl.exe http://127.0.0.1:8080/api/tasks
[]
C:\Users\omimah\Desktop\TaskFlow>
```

التحقق من استعادة خدمة الـ API.

توقف قاعدة البيانات:

تم إيقاف PostgreSQL واختبار النظام، ثم تمت إعادة تشغيل قاعدة البيانات والتحقق من عودة الخدمة.

```
C:\Users\omimah\Desktop\TaskFlow>docker compose stop database
[+] stop 1/1
✓Container taskflow-database-1 Stopped

C:\Users\omimah\Desktop\TaskFlow>curl.exe -i http://127.0.0.1:8080/api/tasks
HTTP/1.1 500 Internal Server Error
Server: nginx/1.31.6
Date: Mon, 28 Sep 2026 02:15:32 GMT
Content-Type: text/plain; charset=utf-8
Content-Length: 21
Connection: keep-alive

Internal Server Error
C:\Users\omimah\Desktop\TaskFlow>
```

اختبار النظام عند توقف قاعدة البيانات

```
C:\Users\omimah\Desktop\TaskFlow>docker compose start database
[+] start 1/1
✓Container taskflow-database-1 Started

C:\Users\omimah\Desktop\TaskFlow>docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS        PORTS
taskflow-api-1      taskflow-api        "uvicorn src.main:ap..." api         36 minutes ago Up 8 minutes 8080/tcp
taskflow-database-1 postgres:16         "docker-entrypoint.s..." database   36 minutes ago Up 17 seconds (healthy) 5432/tcp
taskflow-nginx-1    nginx:alpine        "/docker-entrypoint..." nginx      36 minutes ago Up 36 minutes 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp

C:\Users\omimah\Desktop\TaskFlow>curl.exe http://127.0.0.1:8080/api/tasks
[]
C:\Users\omimah\Desktop\TaskFlow>
```

التحقق من استعادة قاعدة البيانات

إعادة إنشاء خدمة API

تمت إعادة تشغيل خدمة الـ API والتحقق من استمرار عملها والوصول إلى البيانات.

```
C:\Users\omimah\Desktop\TaskFlow>docker compose restart api
[+] restart 0/1
- Container taskflow-api-1 Restarting

C:\Users\omimah\Desktop\TaskFlow>docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS        PORTS
taskflow-api-1      taskflow-api        "uvicorn src.main:ap..." api         56 minutes ago Up 12 seconds 8080/tcp
taskflow-database-1 postgres:16         "docker-entrypoint.s..." database   56 minutes ago Up 20 minutes (healthy) 5432/tcp
taskflow-nginx-1    nginx:alpine        "/docker-entrypoint..." nginx      56 minutes ago Up 56 minutes 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp

C:\Users\omimah\Desktop\TaskFlow>curl.exe http://127.0.0.1:8080/api/tasks
[]
C:\Users\omimah\Desktop\TaskFlow>curl.exe -X POST http://127.0.0.1:8080/api/tasks -H "Content-Type: application/json" -d '{"title":"Recreation Test","description":"Testing API container recreation","status":"pending","id":2}'
{"title":"Recreation Test","description":"Testing API container recreation","status":"pending","id":2}
C:\Users\omimah\Desktop\TaskFlow>
```

اختبار إعادة إنشاء خدمة الـ API

```
C:\Users\omimah\Desktop\TaskFlow>curl.exe http://127.0.0.1:8080/api/tasks
[]
C:\Users\omimah\Desktop\TaskFlow>curl.exe -X POST http://127.0.0.1:8080/api/tasks -H "Content-Type: application/json" -d '{"title":"\Recreation Test\","description":"\pending\","status":"pending","id":2}'
{"title":"Recreation Test","description":"Testing API container recreation","status":"pending","id":2}
C:\Users\omimah\Desktop\TaskFlow>curl.exe http://127.0.0.1:8080/api/tasks
[{"title":"Recreation Test","description":"Testing API container recreation","status":"pending","id":2}]
C:\Users\omimah\Desktop\TaskFlow>
```

التحقق من عمل الـ API بعد إعادة التشغيل.

بناء صورة Docker ونشرها:

تم إنشاء صورة Docker خاصة بخدمة TaskFlow API باستخدام Dockerfile ، مع تثبيت المتطلبات وتشغيل التطبيق داخل الحاوية باستخدام مستخدم غير root .

- تم إصدار الصورة بالنسخة:

v1.0.0

- كما تم نشر الصورة على Docker Hub باستخدام المستودع:

omimah4/tastflow_api

- وتوفير وسمين للصورة:

v1.0.0

latest

```
C:\Users\omimah\Desktop\TaskFlow\api>docker image ls
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
hello-world:latest	5e2309035332	25.9kB	9.49kB	
node-base:latest	fff49f623ad7	408MB	116MB	U
postgres:16	1a6ab3f5345e	642MB	166MB	U
sample-app:latest	b23d895b89d9	408MB	116MB	
taskflow-api:v1.0.0	3adba6bfcc0c	1.96GB	458MB	
todo-app:latest	bf736f7e22cf	501MB	112MB	U
ubuntu:latest	2260313b31c8	160MB	45.3MB	U

```
C:\Users\omimah\Desktop\TaskFlow\api>
```

بناء صورة Docker الخاصة بتطبيق TaskFlow API

```
C:\Users\omimah\Desktop\TaskFlow\api>docker image history taskflow-api:v1.0.0
```

IMAGE	CREATED	CREATED BY	SIZE	COMMENT
3adba6bfcc0c	54 minutes ago	CMD ["uvicorn" "src.main:app" "--host" "0.0.0.0"]	0B	buildkit.dockerfile.v0
<missing>	54 minutes ago	EXPOSE [8080/tcp]	0B	buildkit.dockerfile.v0
<missing>	54 minutes ago	USER appuser	0B	buildkit.dockerfile.v0
<missing>	54 minutes ago	RUN /bin/sh -c useradd --create-home --shell /bin/bash	69.6kB	buildkit.dockerfile.v0
<missing>	54 minutes ago	COPY src ./src # buildkit	49.2kB	buildkit.dockerfile.v0
<missing>	54 minutes ago	RUN /bin/sh -c pip install --no-cache-dir -r requirements.txt	1.37GB	buildkit.dockerfile.v0
<missing>	57 minutes ago	COPY requirements.txt . # buildkit	12.3kB	buildkit.dockerfile.v0
<missing>	57 minutes ago	WORKDIR /app	8.19kB	buildkit.dockerfile.v0
<missing>	8 days ago	CMD ["python3"]	0B	buildkit.dockerfile.v0
<missing>	8 days ago	RUN /bin/sh -c set -eux; for src in idle3 p...	16.4kB	buildkit.dockerfile.v0
<missing>	8 days ago	RUN /bin/sh -c set -eux; savedAptMark="\$(a...	41.4MB	buildkit.dockerfile.v0
<missing>	9 days ago	ENV PYTHON_SHA256=5c8462af5790baf43a321a1559...	0B	buildkit.dockerfile.v0
<missing>	9 days ago	ENV PYTHON_VERSION=3.12.14	0B	buildkit.dockerfile.v0
<missing>	9 days ago	ENV GPG_KEY=7169605F62C751356D054A26A821E680...	0B	buildkit.dockerfile.v0
<missing>	9 days ago	RUN /bin/sh -c set -eux; apt-get update; a...	4.95MB	buildkit.dockerfile.v0
<missing>	9 days ago	ENV LANG=C.UTF-8	0B	buildkit.dockerfile.v0
<missing>	9 days ago	ENV PATH=/usr/local/bin:/usr/local/sbin:/usr...	0B	buildkit.dockerfile.v0
<missing>	10 days ago	# debian.sh --arch 'amd64' out/ 'trixie' '@1...	87.6MB	debuerreotype 0.17

```
C:\Users\omimah\Desktop\TaskFlow\api>
```

عرض طبقات صورة Docker والتحقق من بنائها.

تشغيل النظام باستخدام Docker Compose :

- تم استخدام Docker Compose لتشغيل وإدارة خدمات النظام، والتأكد من صحة إعدادات المشروع قبل التشغيل.
- تم التحقق من إعدادات Compose وتشغيل الخدمات والتأكد من حالتها.

```
C:\Users\omimah\Desktop\TaskFlow>docker compose config
name: taskflow
services:
  api:
    build:
      context: C:\Users\omimah\Desktop\TaskFlow\api
      dockerfile: Dockerfile
    depends_on:
      database:
        condition: service_healthy
        required: true
    environment:
      DATABASE_URL: postgresql://taskflow_user:taskflow_pass@database:5432/taskflow
    networks:
      app-network: null
  database:
    environment:
      POSTGRES_DB: taskflow
      POSTGRES_PASSWORD: taskflow_pass
      POSTGRES_USER: taskflow_user
    healthcheck:
      test:
        - CMD-SHELL
        - pg_isready -U taskflow_user -d taskflow
      timeout: 5s
      interval: 5s
      retries: 5
    image: postgres:16
    networks:
      app-network: null
    volumes:
      - type: volume
        source: db-data
        target: /var/lib/postgresql/data
        volume: {}
  nginx:
    depends_on:
      api:
        condition: service_started
        required: true
    image: nginx:alpine
    networks:
      app-network: null
    ports:
      - mode: ingress
        target: 80
        published: "8080"
        protocol: tcp
    volumes:
      - type: bind
        source: C:\Users\omimah\Desktop\TaskFlow\nginx\nginx.conf
        target: /etc/nginx/nginx.conf
        read_only: true
        bind: {}
networks:
  app-network:
    name: taskflow_app-network
    driver: bridge
volumes:
  db-data:
    name: taskflow_db-data
    Windows تنشيط
    انتقل إلى الإعدادات لتنشيط Windows
C:\Users\omimah\Desktop\TaskFlow>
```

التحقق النهائي من إعدادات Docker Compose .

```
C:\Users\omimah\Desktop\TaskFlow>docker compose up -d --build
[+] up 11/11
  ✓ Image nginx:alpine Pulled
#1 [internal] load local bake definitions
#1 reading from stdin 525B done
#1 DONE 0.0s

#2 [internal] load build definition from Dockerfile
#2 transferring dockerfile: 335B done
#2 DONE 0.0s

#3 [internal] load metadata for docker.io/library/python:3.12-slim
#3 DONE 1.4s

#4 [internal] load .dockerignore
#4 transferring context: 96B done
#4 DONE 0.0s

#5 [1/6] FROM docker.io/library/python:3.12-slim@sha256:f77ac9e44ae96ef2c90b8053ea08c31f8be030f824196b0ae4db6d462c84e51f
#5 resolve docker.io/library/python:3.12-slim@sha256:f77ac9e44ae96ef2c90b8053ea08c31f8be030f824196b0ae4db6d462c84e51f 0.0s done
#5 DONE 0.0s

#6 [internal] load build context
#6 transferring context: 462B done
#6 DONE 0.0s

#7 [2/6] WORKDIR /app
#7 CACHED

#8 [4/6] RUN pip install --no-cache-dir -r requirements.txt
#8 CACHED

#9 [3/6] COPY requirements.txt .
#9 CACHED

#10 [5/6] COPY src ./src
#10 CACHED

#11 [6/6] RUN useradd --create-home --shell /bin/bash appuser
#11 CACHED

#12 exporting to image
#12 exporting layers done
#12 exporting manifest sha256:f038321c25354c58d8f35e7d1da5979d27914ce6397d2ca36719f5df01eda059 0.0s done
#12 exporting config sha256:d881775eafd0ba246316bd51afd0e0121895bccf77a24a749a88093bace1064 0.0s done
#12 exporting attestation manifest sha256:21df2558dcd3a8d761345a804eed9150570f6f248fa028d19b680fd857a72ca2 0.0s done
#12 exporting manifest list sha256:c6146bfeb75a2adc0efee9ce72a68079a94ab53e1c4e82e83124e89b4a095a97 0.0s done
#12 naming to docker.io/library/taskflow-api:latest done
#12 unpacking to docker.io/library/taskflow-api:latest done
#12 DONE 0.1s

[+] up 17/17g provenance for metadata file
  ✓ Image nginx:alpine Pulled
  ✓ Image taskflow-api Built
  ✓ Network taskflow_app-network Created
  ✓ Volume taskflow_db-data Created
  ✓ Container taskflow-database-1 Healthy
  ✓ Container taskflow-api-1 Started
  ✓ Container taskflow-nginx-1 Started

What's next:
  Filter, search, and stream logs from all your Compose services
  in one place with Docker Desktop's Logs view. docker-desktop:///dashboard/logs?appId=taskflow

C:\Users\omimah\Desktop\TaskFlow>
```

تشغيل خدمات النظام باستخدام Docker Compose

```
C:\Users\omimah\Desktop\TaskFlow>docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS                PORTS
taskflow-api-1      taskflow-api         "uvicorn src.main:ap-  api        12 seconds ago Up 11 seconds        8080/tcp
taskflow-database-1 postgres:16          "docker-entrypoint.s- database   10 minutes ago Up 10 minutes (healthy)  5432/tcp
taskflow-nginx-1    nginx:alpine         "/docker-entrypoint..." nginx      10 minutes ago Up 10 minutes        0.0.0.0:8080->80/tcp, [::]:8080->80/tcp

C:\Users\omimah\Desktop\TaskFlow>
```

التحقق من حالة الخدمات بعد التشغيل

نتائج المشروع:

أظهرت الاختبارات أن نظام TaskFlow API يعمل بصورة صحيحة ضمن بيئة Docker ، وتم التحقق من:

-تشغيل الخدمات الثلاث Nginx و FastAPI و PostgreSQL.

- تنفيذ عمليات CRUD بنجاح.

- اتصال الـ API بقاعدة البيانات.

- عزل منافذ الـ API وقاعدة البيانات عن الجهاز المضيف.

- استمرار البيانات باستخدام Named Volume.
 - استعادة النظام بعد توقف الـ API أو قاعدة البيانات.
 - تشغيل صورة Docker بإصدار محدد ونشرها على Docker Hub.
- وبذلك تم تحقيق المتطلبات الأساسية لتشغيل نظام حاويات متكامل وقابل لإعادة التشغيل والنشر.

```
C:\Users\omimah\Desktop\TaskFlow>curl http://127.0.0.1:8080/api/tasks
[{"title":"TaskFlow Test","description":"Testing Nginx reverse proxy","status":"in_progress","id":1}]
C:\Users\omimah\Desktop\TaskFlow>
```

النتيجة النهائية لاختبار استمرار البيانات

الخاتمة:

تم في هذا المشروع تطوير TaskFlow API وتطبيق مفاهيم الحوسبة السحابية والحاويات باستخدام Docker و Docker Compose .

اعتمد النظام على Nginx و FastAPI و PostgreSQL مع تطبيق عزل الخدمات، واستمرارية البيانات، وتشغيل الـ API باستخدام مستخدم غير root .

كما تم اختبار عمليات الـ API وحالات الفشل والاستعادة، ونشر صورة التطبيق على Docker Hub بإصدار محدد، مما يجعل النظام قابلاً لإعادة التشغيل والنشر بطريقة منظمة.